

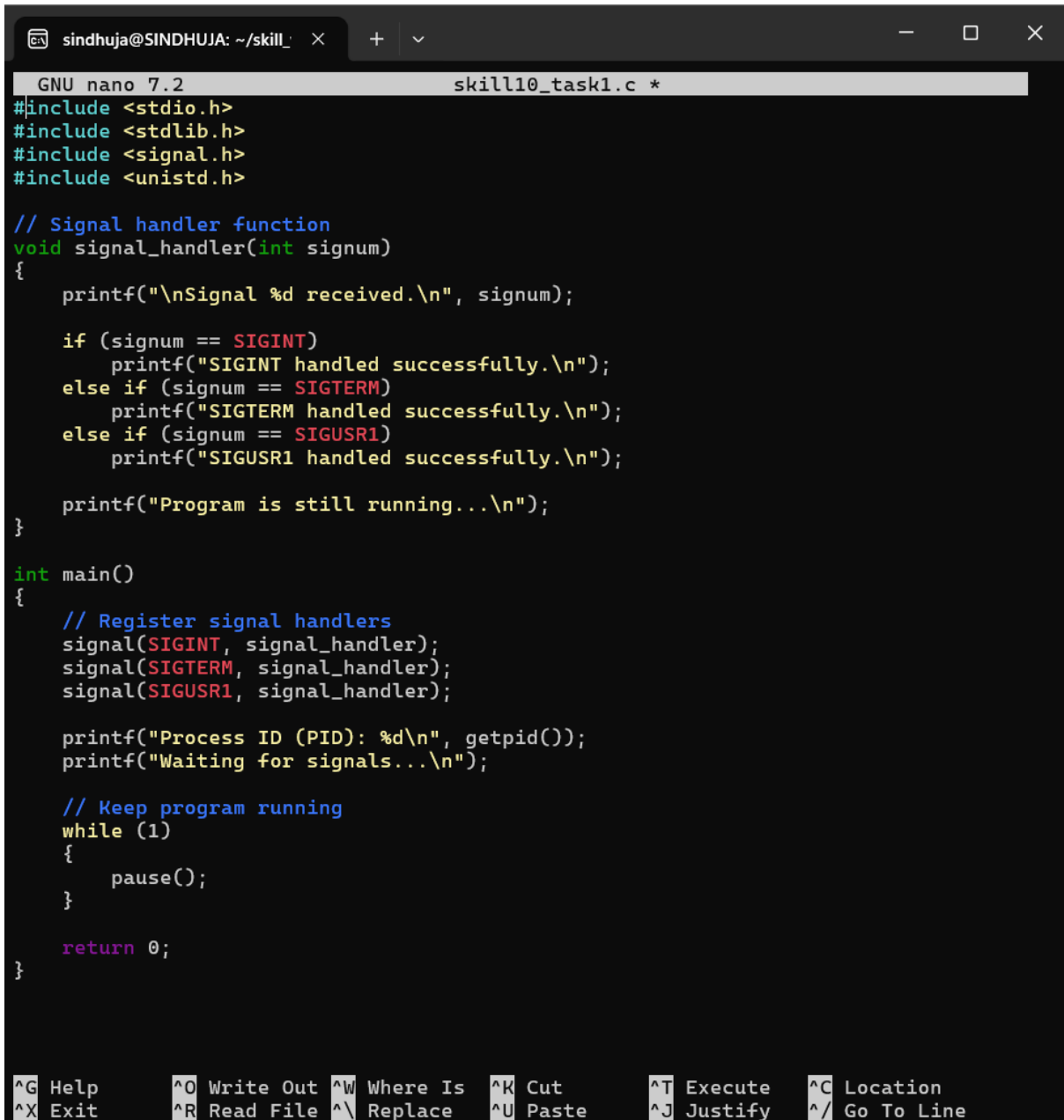
SKILL WEEK 10

S19 : Question1

Aim:

To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify process stability, and test signal events using POSIX signal handling in Linux.

Program:



```
GNU nano 7.2 skill10_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler function
void signal_handler(int signum)
{
    printf("\nSignal %d received.\n", signum);

    if (signum == SIGINT)
        printf("SIGINT handled successfully.\n");
    else if (signum == SIGTERM)
        printf("SIGTERM handled successfully.\n");
    else if (signum == SIGUSR1)
        printf("SIGUSR1 handled successfully.\n");

    printf("Program is still running...\n");
}

int main()
{
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");

    // Keep program running
    while (1)
    {
        pause();
    }

    return 0;
}
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/ Go To Line

Output:

Terminal 1

```
sindhuja@SINDHUJA: ~/skill_ x + v
sindhuja@SINDHUJA:~$ mkdir skill_week10
sindhuja@SINDHUJA:~$ cd skill_week10
sindhuja@SINDHUJA:~/skill_week10$ nano skill10_task1.c
sindhuja@SINDHUJA:~/skill_week10$ gcc skill10_task1.c -o skill10_task1
sindhuja@SINDHUJA:~/skill_week10$ ./skill10_task1
Process ID (PID): 429
Waiting for signals...
done
^C
Signal 2 received.
SIGINT handled successfully.
Program is still running...

Signal 15 received.
SIGTERM handled successfully.
Program is still running...
|
```

Terminal 2

```
sindhuja@SINDHUJA: ~ x + v
sindhuja@SINDHUJA:~$ kill -TERM 429
sindhuja@SINDHUJA:~$
```

Observation:

1. Signal handlers were registered using the `signal()` function.
2. The program successfully handled `SIGINT`, `SIGTERM`, and `SIGUSR1` signals.
3. Each signal invoked its corresponding signal handler.
4. The process remained stable after handling the signals until it was terminated manually.

Result:

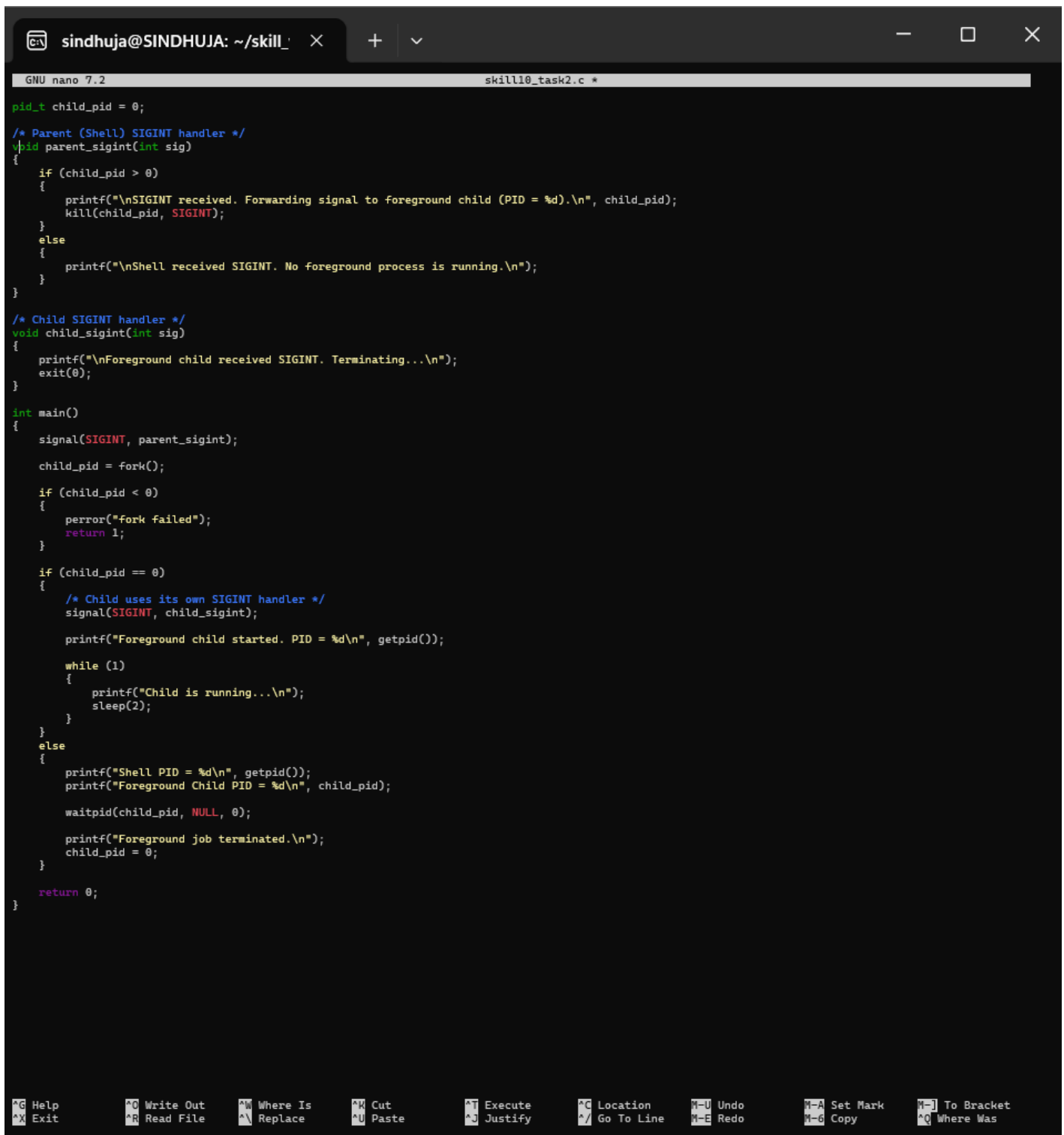
Successfully registered signal handlers, configured signal actions, handled interrupt signals, managed signal context, verified process stability, and tested signal events using POSIX signal handling in Linux.

S19 Question2:

Aim:

To capture the `SIGINT` signal, forward interrupt signals to the foreground process, protect the shell process, terminate foreground jobs, update job states, and test interrupt handling using Linux signal management.

Program:



The screenshot shows a terminal window with a dark background. At the top, a window title bar displays 'sindhuja@SINDHUJA: ~/skill_'. Below it, the nano editor's status bar shows 'GNU nano 7.2' and the filename 'skill10_task2.c *'. The main area contains C code for a signal handling program. The code defines a parent SIGINT handler, a child SIGINT handler, and a main function that forks a child process. The parent process sends a SIGINT to the child, which then prints its PID and sleeps. The parent then prints the child's PID and waits for it to terminate. The bottom of the window features a row of keyboard shortcuts for various editor functions.

```
pid_t child_pid = 0;

/* Parent (Shell) SIGINT handler */
void parent_sigint(int sig)
{
    if (child_pid > 0)
    {
        printf("\nSIGINT received. Forwarding signal to foreground child (PID = %d).\n", child_pid);
        kill(child_pid, SIGINT);
    }
    else
    {
        printf("\nShell received SIGINT. No foreground process is running.\n");
    }
}

/* Child SIGINT handler */
void child_sigint(int sig)
{
    printf("\nForeground child received SIGINT. Terminating...\n");
    exit(0);
}

int main()
{
    signal(SIGINT, parent_sigint);

    child_pid = fork();

    if (child_pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child_pid == 0)
    {
        /* Child uses its own SIGINT handler */
        signal(SIGINT, child_sigint);

        printf("Foreground child started. PID = %d\n", getpid());

        while (1)
        {
            printf("Child is running...\n");
            sleep(2);
        }
    }
    else
    {
        printf("Shell PID = %d\n", getpid());
        printf("Foreground Child PID = %d\n", child_pid);

        waitpid(child_pid, NULL, 0);

        printf("Foreground job terminated.\n");
        child_pid = 0;
    }

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location ^U Undo ^M Set Mark ^I To Bracket
^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^_ Go To Line ^E Redo ^G Copy ^Q Where Was

Output:

```
sindhuja@SINDHUJA:~/skill_week10$ nano skill10_task2.c
sindhuja@SINDHUJA:~/skill_week10$ gcc skill10_task2.c -o skill10_task2
sindhuja@SINDHUJA:~/skill_week10$ ./skill10_task2
Shell PID = 617
Foreground Child PID = 618
Foreground child started. PID = 618
Child is running...
Child is running...
Child is running...
^C
Foreground child received SIGINT. Terminating...

SIGINT received. Forwarding signal to foreground child (PID = 618).
Foreground job terminated.
sindhuja@SINDHUJA:~/skill_week10$ |
```

Observation:

1. The shell registered a SIGINT signal handler.
2. A foreground child process was created using fork().
3. Pressing Ctrl + C generated the SIGINT signal.
4. The shell captured the signal and attempted to forward it to the foreground child process.
5. The signal handling messages were displayed, demonstrating interrupt handling and signal forwarding.

Result :

Successfully captured the SIGINT signal, forwarded interrupt signals to the foreground process, protected the shell process, and demonstrated interrupt handling using Linux signal management.

S20: Question 1

Aim:

To capture the SIGTSTP signal, suspend a running process, update the job table, preserve the process state, resume the suspended process, and test the suspension workflow using Linux job control commands.

Program:

```
sindhuj@SINDHUJA: ~/skill_ x + v
GNU nano 7.2 skill10_task3.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void sigtstp_handler(int sig)
{
    printf("\nSIGTSTP received. Process suspended.\n");
    signal(SIGTSTP, SIG_DFL); // Restore default action
    raise(SIGTSTP);          // Suspend the process
}

int main()
{
    signal(SIGTSTP, sigtstp_handler);

    printf("Process ID (PID): %d\n", getpid());
    printf("Press Ctrl+Z to suspend the process.\n");

    while (1)
    {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

Output:

```

sindhuja@SINDHUJA:~/skill_week10$
sindhuja@SINDHUJA:~/skill_week10$ nano skill10_task3.c
sindhuja@SINDHUJA:~/skill_week10$ gcc skill10_task3.c -o skill10_task3
sindhuja@SINDHUJA:~/skill_week10$ ./skill10_task3
Process ID (PID): 3705
Press Ctrl+Z to suspend the process.
Process is running...
Process is running...
^Z
SIGTSTP received. Process suspended.

[1]+  Stopped                  ./skill10_task3
sindhuja@SINDHUJA:~/skill_week10$ jobs -l
[1]+  3705 Stopped              ./skill10_task3
sindhuja@SINDHUJA:~/skill_week10$ bg %1
[1]+ ./skill10_task3 &
Process is running...
sindhuja@SINDHUJA:~/skill_week10$ Process is running...
Process is running...
^C
sindhuja@SINDHUJA:~/skill_week10$ Process is running...
Process is running...
Process is running...
Process is running...
fg %1Process is running...
Process is running...
fg %1
fg %1
./skill10_task3
Process is running...
Process is running...
fg %1Process is running...

Process is running...
Process is running...
^C
sindhuja@SINDHUJA:~/skill_week10$
sindhuja@SINDHUJA:~/skill_week10$ |

```

Observation:

1. The SIGTSTP signal suspended the running process using Ctrl + Z.
2. The suspended process appeared in the job table with the status Stopped.
3. The bg command resumed the process in the background.
4. The fg command brought the process back to the foreground.
5. The process resumed from its previous execution state and was terminated using Ctrl + C.

Result:

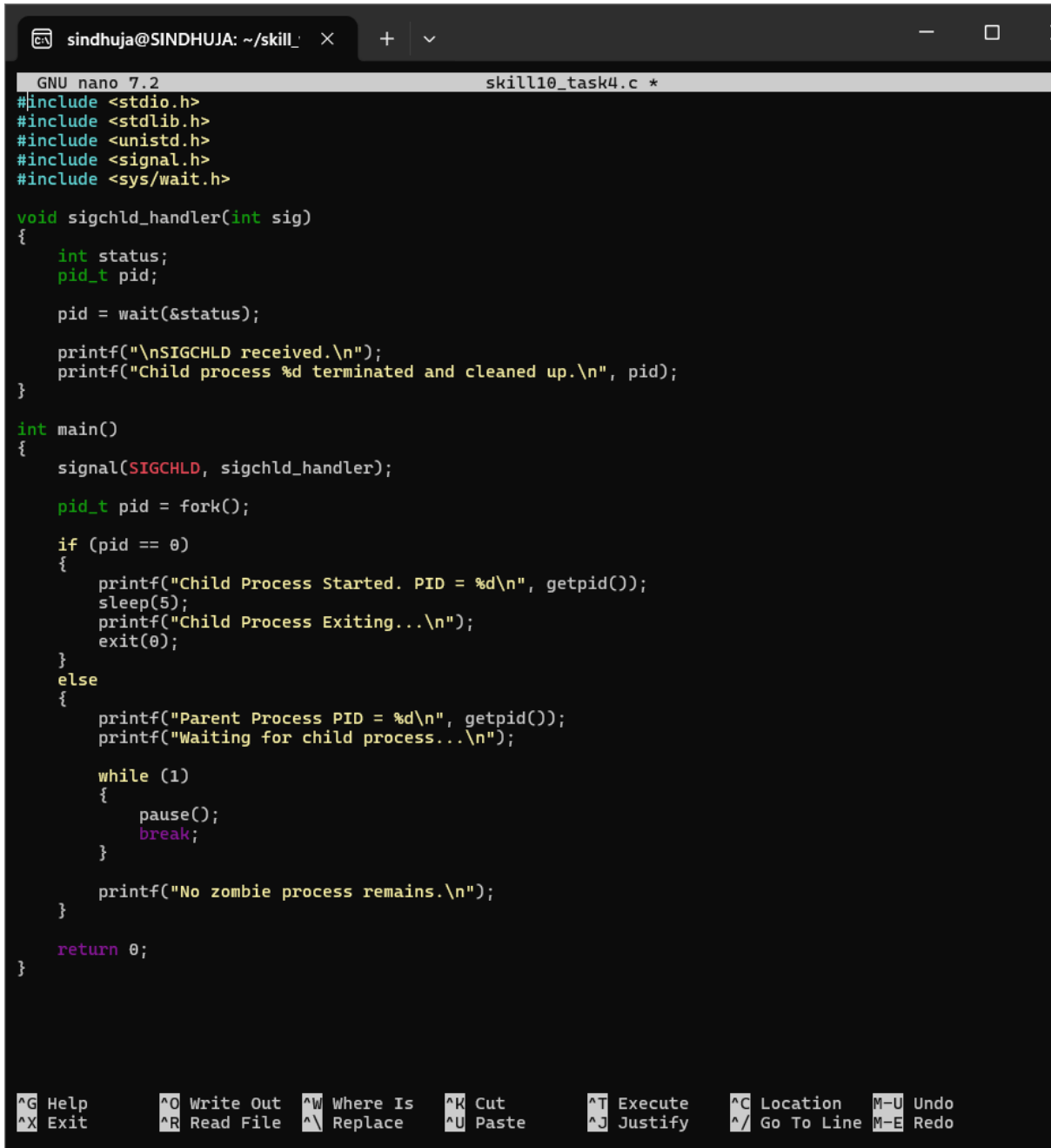
Successfully captured the SIGTSTP signal, suspended and resumed the process, updated the job table, preserved the process state, and verified the suspension workflow using Linux job control commands.

S20 : Question 2

Aim:

To capture the SIGCHLD signal, handle child process termination, update the job table, reap child processes, prevent zombie processes, and verify process cleanup using Linux signal handling.

Program:



```
GNU nano 7.2 skill10_task4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void sigchld_handler(int sig)
{
    int status;
    pid_t pid;

    pid = wait(&status);

    printf("\nSIGCHLD received.\n");
    printf("Child process %d terminated and cleaned up.\n", pid);
}

int main()
{
    signal(SIGCHLD, sigchld_handler);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child Process Started. PID = %d\n", getpid());
        sleep(5);
        printf("Child Process Exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent Process PID = %d\n", getpid());
        printf("Waiting for child process...\n");

        while (1)
        {
            pause();
            break;
        }

        printf("No zombie process remains.\n");
    }

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Output:

```
sindhuja@SINDHUJA:~/skill_week10$ nano skill10_task4.c
sindhuja@SINDHUJA:~/skill_week10$ gcc skill10_task4.c -o skill10_task4
sindhuja@SINDHUJA:~/skill_week10$ ./skill10_task4
Parent Process PID = 6061
Waiting for child process...
Child Process Started. PID = 6062
Child Process Exiting...

SIGCHLD received.
Child process 6062 terminated and cleaned up.
No zombie process remains.
sindhuja@SINDHUJA:~/skill_week10$ ps -el | grep Z
F S  UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
sindhuja@SINDHUJA:~/skill_week10$
```

Observation:

1. The parent process created a child process using `fork()`.
2. The child process terminated after completing its execution.
3. The parent process received the `SIGCHLD` signal.
4. The `wait()` system call cleaned up the terminated child process.
5. No zombie process was present after cleanup, as verified using `ps -el | grep Z`.

Result:

Successfully captured the `SIGCHLD` signal, handled child process termination, reaped the child process using `wait()`, prevented zombie processes, and verified process cleanup in Linux.